

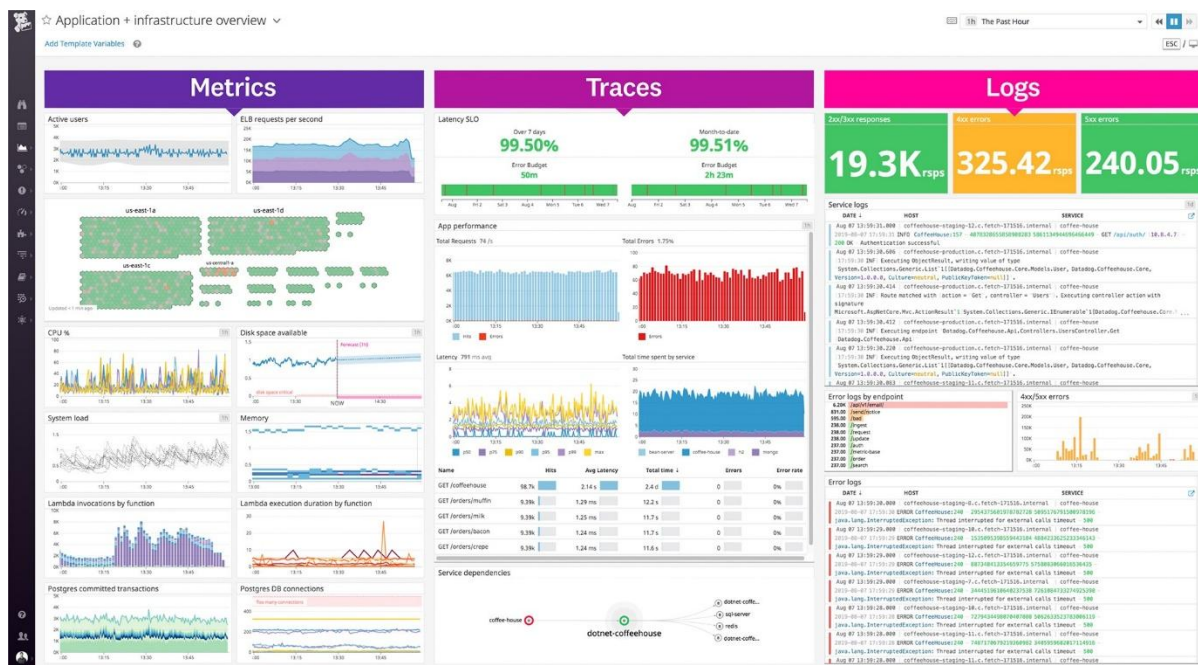
Monitoring Machine Learning Systems

Intuition

Even though we've trained and thoroughly evaluated our model, the real work begins once we deploy to production. This is one of the fundamental differences between traditional software engineering and ML development. Traditionally, with rule based, deterministic, software, most of the work occurs at the initial stage and once deployed, our system works as we've defined it. But with machine learning, we haven't explicitly defined how something works but used data to architect a probabilistic solution. This approach is subject to natural performance degradation over time, as well as unintended behavior, since the data exposed to the model will be different from what it has been trained on. This isn't something we should be trying to avoid but rather understand and mitigate as much as possible. In this lesson, we'll understand the shortcomings from attempting to capture performance degradation to motivate the need for drift detection.

System health

The first step to ensure that our model is performing well is to ensure that the actual system is up and running as it should. This can include metrics specific to service requests such as latency, throughput, error rates, etc. as well as infrastructure utilization such as CPU/GPU utilization, memory, etc.



Fortunately, most cloud providers and even orchestration layers will provide this insight into our system's health for free through a dashboard. In the event we don't, we can easily use [Grafana](#), [Datadog](#), etc. to ingest system performance metrics from logs to create a customized dashboard and set alerts.

Performance

Unfortunately, just monitoring the system's health won't be enough to capture the underlying issues with our model. So, naturally, the next layer of metrics to monitor involves the model's performance. These could be quantitative evaluation metrics that we used during model evaluation (accuracy, precision, f1, etc.) but also key business metrics that the model influences (ROI, click rate, etc.).

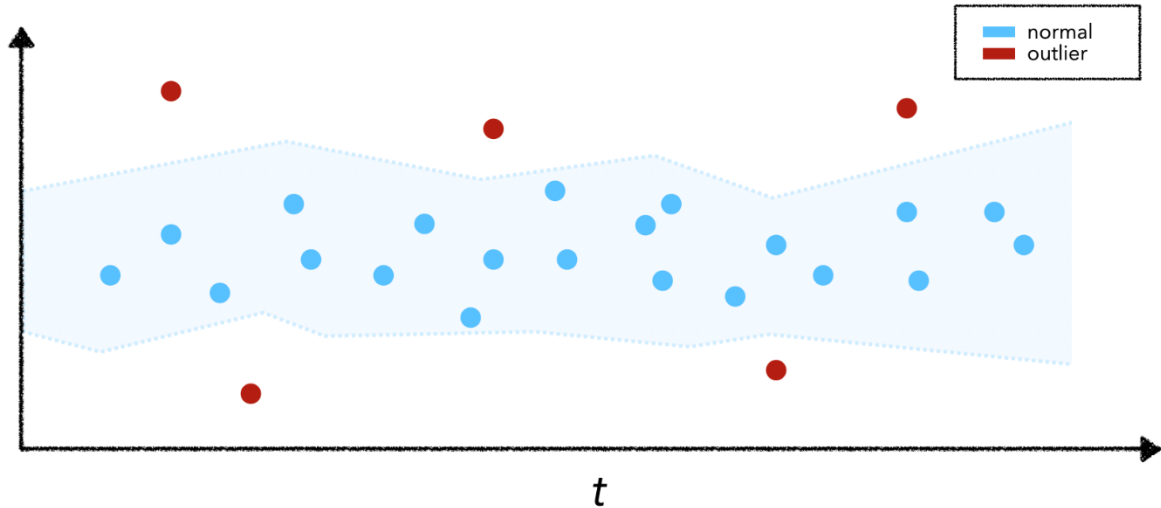
It's usually never enough to just analyze the cumulative performance metrics across the entire span of time since the model has been deployed. Instead, we should also inspect performance over a period that's significant for our application (ex. daily). These sliding metrics might be more indicative of our system's health, and we might be able to identify issues faster by not obscuring them with historical data.

Follow along interactive notebook in the monitoring-ml repository as we implement the concepts below.

<https://github.com/badrhr/Monitoring-ML>

Outliers

With drift, we're comparing a window of production data with reference data as opposed to looking at any one specific data point. While each individual point may not be an anomaly or outlier, the group of points may cause a drift. The easiest way to illustrate this is to imagine feeding our live model the same data point repeatedly. The actual data point may not have anomalous features but feeding it repeatedly will cause the feature distribution to change and lead to drift.



Unfortunately, it's not very easy to detect outliers because it's hard to constitute the criteria for an outlier. Therefore, the outlier detection task is typically unsupervised and requires a stochastic streaming algorithm to identify potential outliers. Luckily, there are several powerful libraries such as [PyOD](#), [Alibi Detect](#), [WhyLogs](#) (uses [Apache DataSketches](#)), etc. that offer a suite of outlier detection functionality (largely for tabular and image data for now).

Typically, outlier detection algorithms fit (ex. via reconstruction) to the training set to understand what normal data looks like and then we can use a threshold to predict outliers. If we have a small, labeled dataset with outliers, we can empirically choose our threshold but if not, we can choose some reasonable tolerance.

```
from alibi_detect.od import OutlierVAE
X_train = (n_samples, n_features)
outlier_detector = OutlierVAE(
    threshold=0.05,
    encoder_net=encoder,
    decoder_net=decoder,
    latent_dim=512
)
outlier_detector.fit(X_train, epochs=50)
outlier_detector.infer_threshold(X, threshold_perc=95) # infer from % outliers
preds = outlier_detector.predict(X, outlier_type="instance", outlier_perc=75)
```

When we identify outliers, we may want to let the end user know that the model's response may not be reliable. Additionally, we may want to remove the outliers from the next training set or further inspect them and upsample them in case there early signs of what future distributions of incoming features will look like.

Solutions

It's not enough to just be able to measure drift or identify outliers but to also be able to act on it. We want to be able to alert on drift, inspect it and then act on it.

Alert

Once we've identified outliers and/or measured statistically significant drift, we need to devise a workflow to notify stakeholders of the issues. A negative connotation with monitoring is fatigue stemming from false positive alerts. This can be mitigated by choosing the appropriate constraints (ex. alerting thresholds) based on what's important to our specific application. For example, thresholds could be:

- fixed values/range for situations where we're concretely aware of expected upper/lower bounds.

```
if percentage_unk_tokens > 5%:  
    trigger_alert()
```

- **forecasted** thresholds dependent on previous inputs, time, etc.

```
if current_f1 < forecast_f1(current_time):  
    trigger_alert()
```

- appropriate p-values for different drift detectors ($\downarrow$ p-value = $\uparrow$ confident that the distributions are different).

```
from alibi_detect.cd import KSDrift  
length_drift_detector = KSDrift(reference, p_val=0.01)
```

Once we have our carefully crafted alerting workflows in place, we can notify stakeholders as issues arise via email, [Slack](#), [PageDuty](#), etc. The stakeholders can be of various levels (core engineers, managers, etc.) and they can subscribe to the alerts that are relevant for them.

Inspect

Once we receive an alert, we need to inspect it before acting on it. An alert needs several components in order for us to completely inspect it:

- specific alert that was triggered
- relevant metadata (time, inputs, outputs, etc.)
- thresholds / expectations that failed
- drift detection tests that were conducted
- data from reference and test windows

- `log` records from the relevant window of time

```
# Sample alerting ticket
{
  "triggered_alerts": ["text_length_drift"],
  "threshold": 0.05,
  "measurement": "KSDrift",
  "distance": 0.86,
  "p_val": 0.03,
  "reference": [],
  "target": [],
  "logs": ...
}
```

With these pieces of information, we can work backwards from the alert towards identifying the root cause of the issue. **Root cause analysis (RCA)** is an important first step when it comes to monitoring because we want to prevent the same issue from impacting our system again. Often times, many alerts are triggered but they maybe all actually be caused by the same underlying issue. In this case, we'd want to intelligently trigger just one alert that pinpoints the core issue. For example, let's say we receive an alert that our overall user satisfaction ratings are reducing but we also receive another alert that our North American users also have low satisfaction ratings. Here's the system would automatically assess for drift in user satisfaction ratings across many different slices and aggregations to discover that only users in a specific area are experiencing the issue but because it's a popular user base, it ends up triggering all aggregate downstream alerts as well!

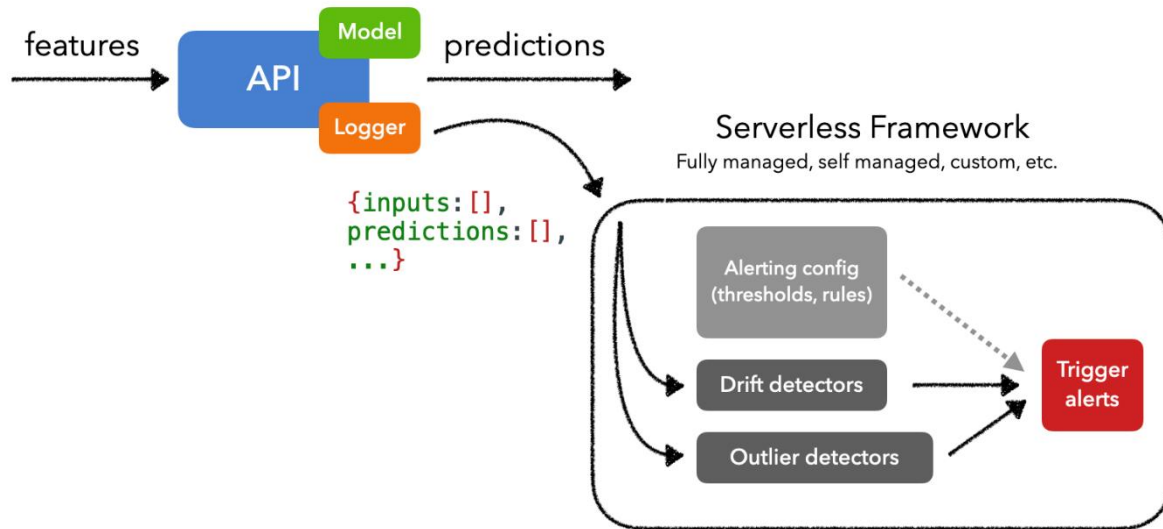
Act

There are many different ways we can act to drift based on the situation. An initial impulse may be to retrain our model on the new data but it may not always solve the underlying issue.

- ensure all data expectations have passed.
- confirm no data schema changes.
- retrain the model on the new shifted dataset.
- move the reference window to more recent data or give it more weight.
- determine if outliers are potentially valid data points.

Production

Since detecting drift and outliers can involve computing intensive operations, we need a solution that can execute serverless workloads on top of our event data streams (ex. [Kafka](#)). Typically, these solutions will ingest payloads (ex. model's inputs and outputs) and can trigger monitoring workloads. This allows us to segregate the resources for monitoring from our actual ML application and scale them as needed.



When it comes to implementing a monitoring system, we have several options, ranging from fully managed to from-scratch. Several popular managed solutions are [Arize](#), [Arthur](#), [Fiddler](#), [Gantry](#), [Mona](#), [WhyLabs](#), etc., all of which allow us to create custom monitoring views, trigger alerts, etc. There are even several great open-source solutions such as [EvidentlyAI](#), [TorchDrift](#), [WhyLogs](#), etc.

We'll often notice that monitoring solutions are offered as part of the larger deployment option such as [Sagemaker](#), [TensorFlow Extended \(TFX\)](#), [TorchServe](#), etc. And if we're already working with Kubernetes, we could use [KNative](#) or [Kubeless](#) for serverless workload management. But we could also use a higher-level framework such as [KFServing](#) or [Seldon core](#) that natively use a serverless framework like KNative.

References

- [An overview of unsupervised drift detection methods](#)
- [Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift](#)
- [Monitoring and explainability of models in production](#)
- [Detecting and Correcting for Label Shift with Black Box Predictors](#)
- [Outlier and anomaly pattern detection on data streams](#)